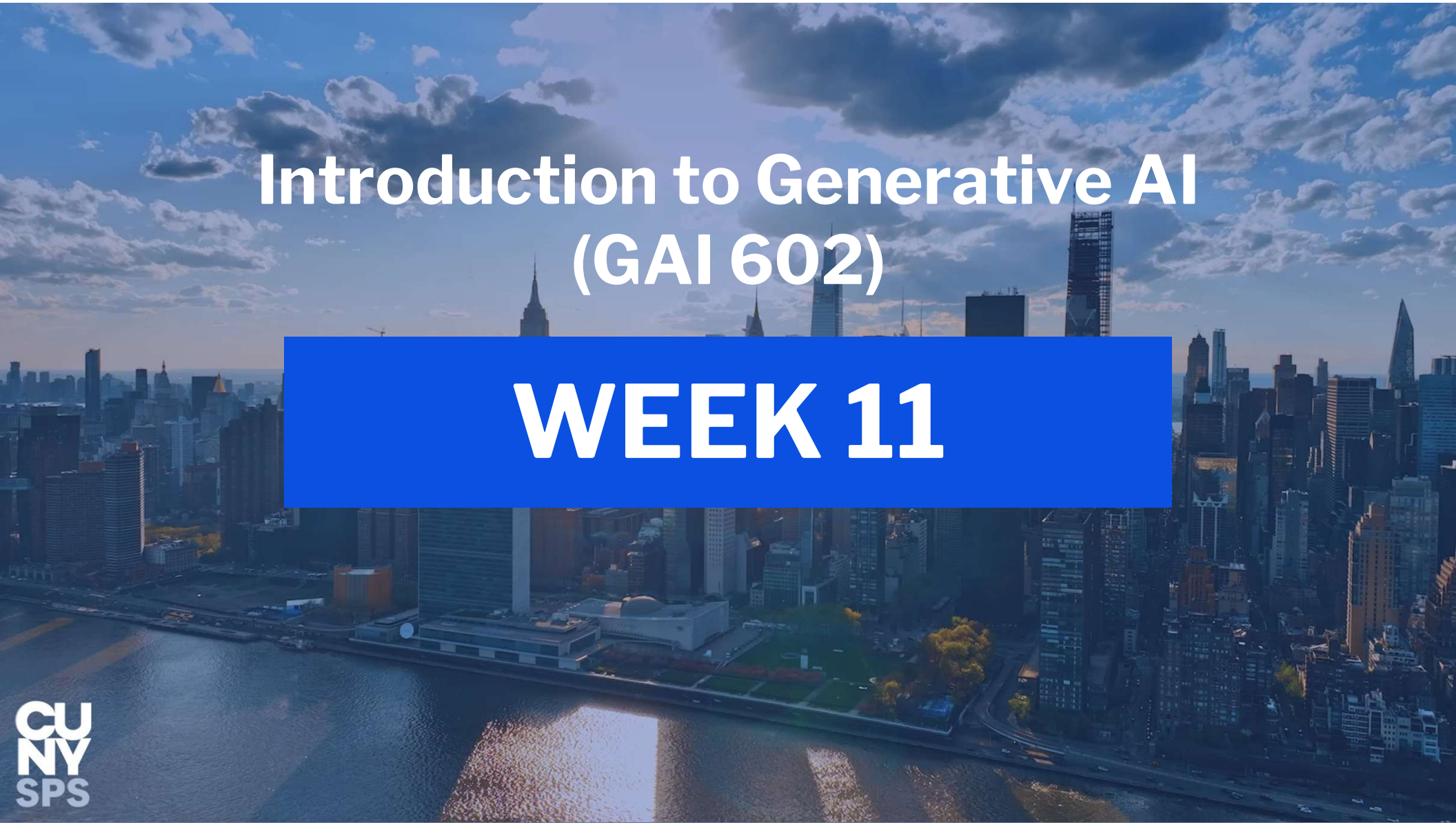

Introduction to Generative AI
(GAI 602)
WEEK 11
CUNY
SPS

CUNY SPS Data Science Seminar

6:30PM Thursday, November 19th

CUNY@Amazon, Lobby 12 West 39th St. NY, NY 10018

AJ Strauman-Scott

Renaissance Philanthropy

Small Voices, Big Challenges: Automatic Speech Recognition for Scaling Remote One-on-One Tutoring



CUNY THE CITY UNIVERSITY OF NEW YORK

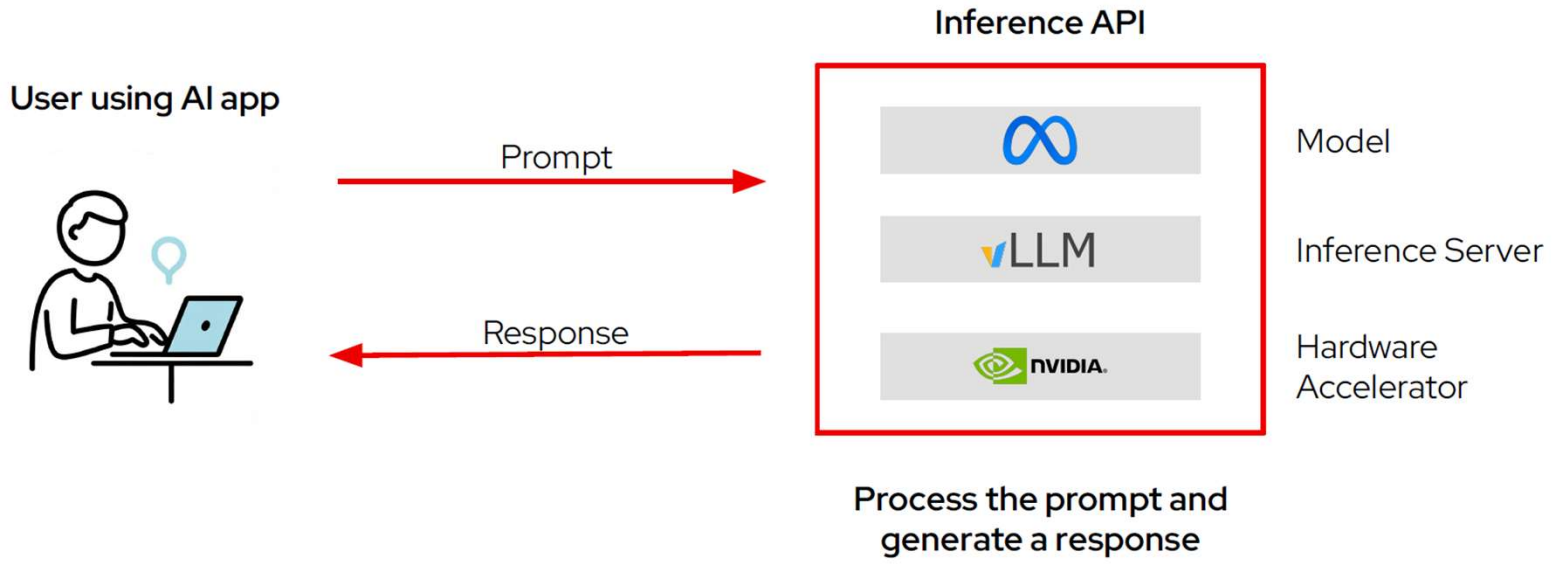
What does applied machine learning look like when the goal is to accelerate student learning? Using one-on-one tutoring as a case study, this talk follows the path from recorded conversations to transcripts, analysis, and monitoring. Along the way, we'll examine the challenges of recognizing children's speech, how transcription errors affect what we can reliably learn about students' understanding and tutors' support, and what it takes to make these pipelines reliable in real-world educational settings.

https://us02web.zoom.us/meeting/register/vNnES24tQkqUktS86HL1og


Model Inference

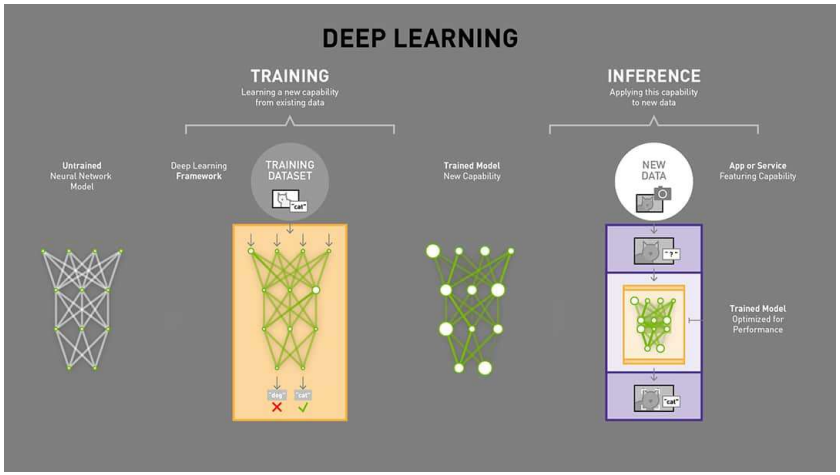
Inference

Inference is the process of using a trained model to generate new text in response to a user prompt.



Source: Redhat

Training vs. Inference



Unlike training, inference is the process of using a trained model to make predictions or generate outputs for unseen data.

This is the “deployment” stage where models provide value, powering applications like chatbots, recommendation systems, and image generation.

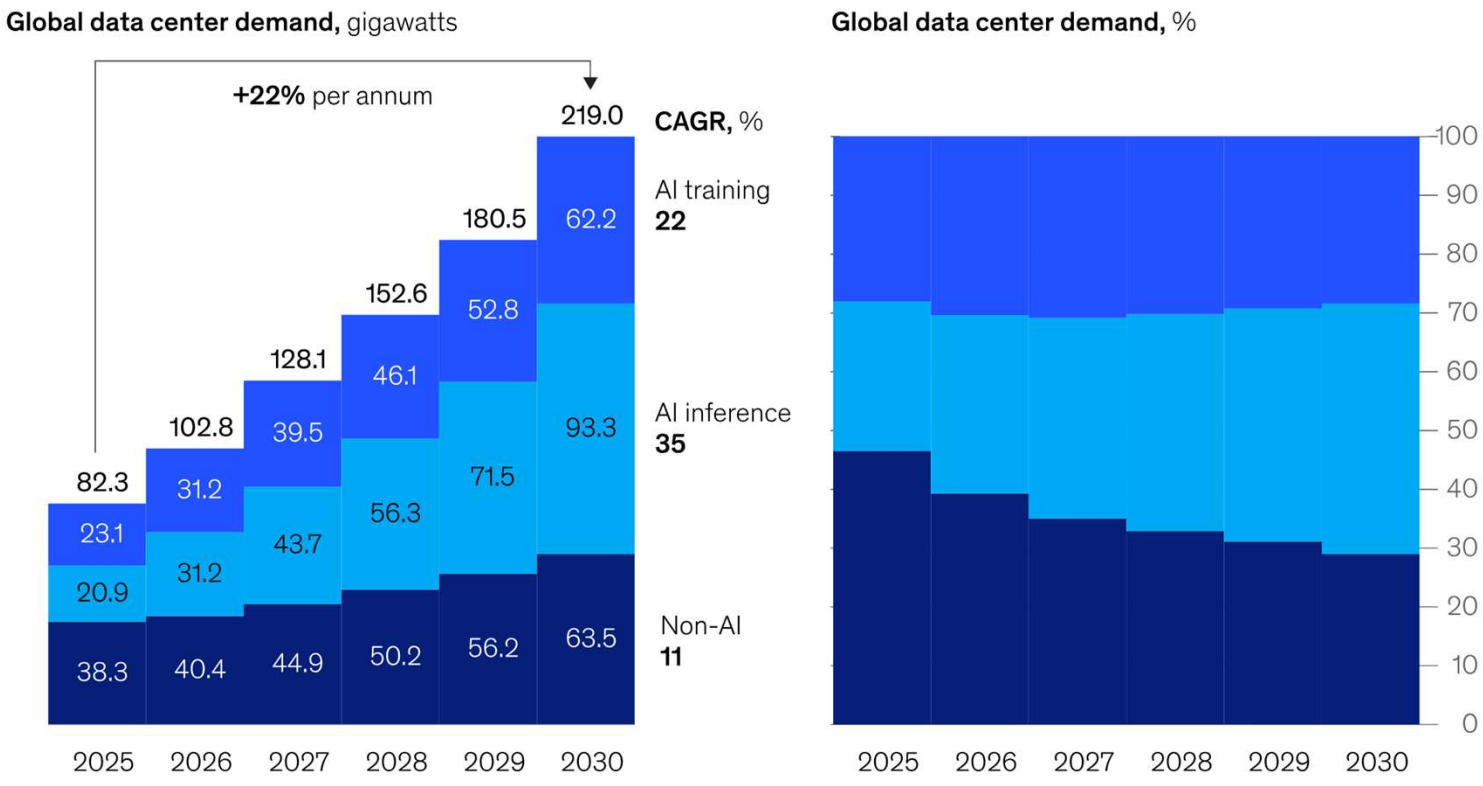
Characteristics of inference:

1. **Latency Sensitivity**
   Critical for applications like chatbots or live translation. Performance is measured in milliseconds per query.
2. **Resource Efficiency**
   Must run on diverse hardware, from high-end GPUs to edge devices. Optimization often trades accuracy for speed.
3. **Predictability**
   Unlike training, inference workloads are more predictable and involve fixed computations.

Source: Nvidia

Inference Cost

Inference workloads could make up more than 40 percent of data center demand in 2030, growing 35 percent CAGR until 2030.



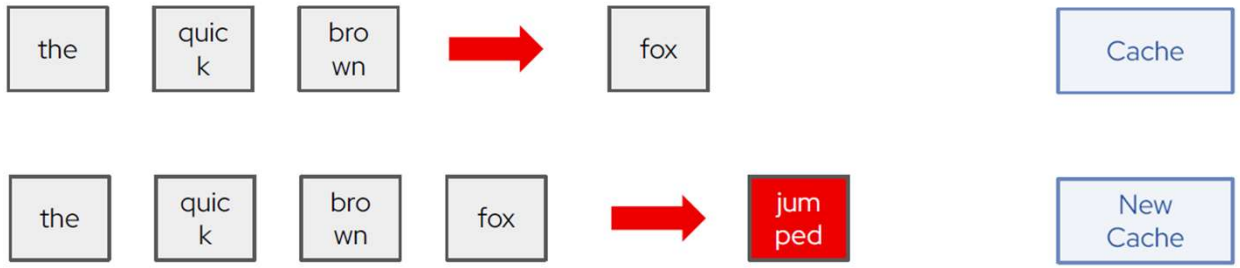
Note: Includes all provider types.

Source: McKinsey

Key optimization: caching

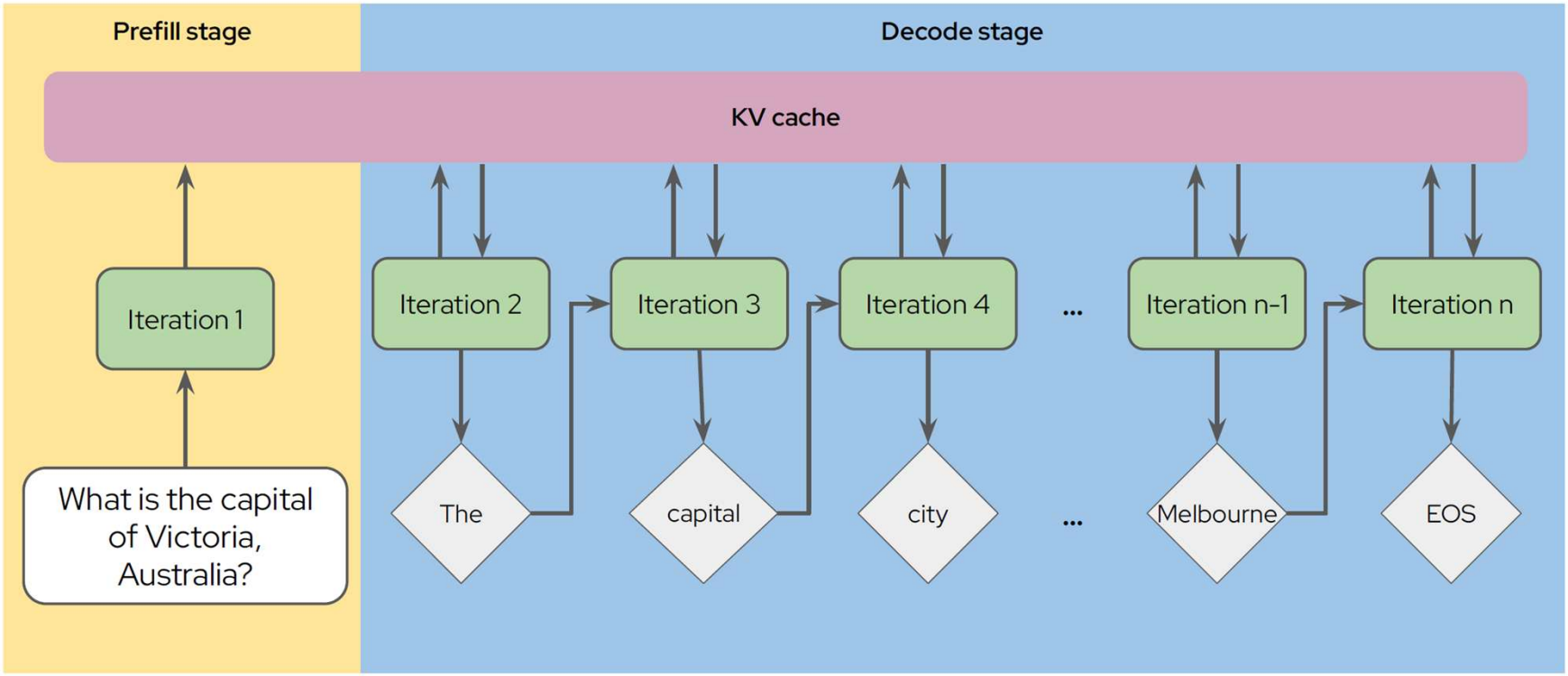
Processing all previous tokens is expensive

We can “re-use” the intermediate states between iterations to “skip” processing all previous tokens



Source: Redhat

Exploiting the transformer architecture



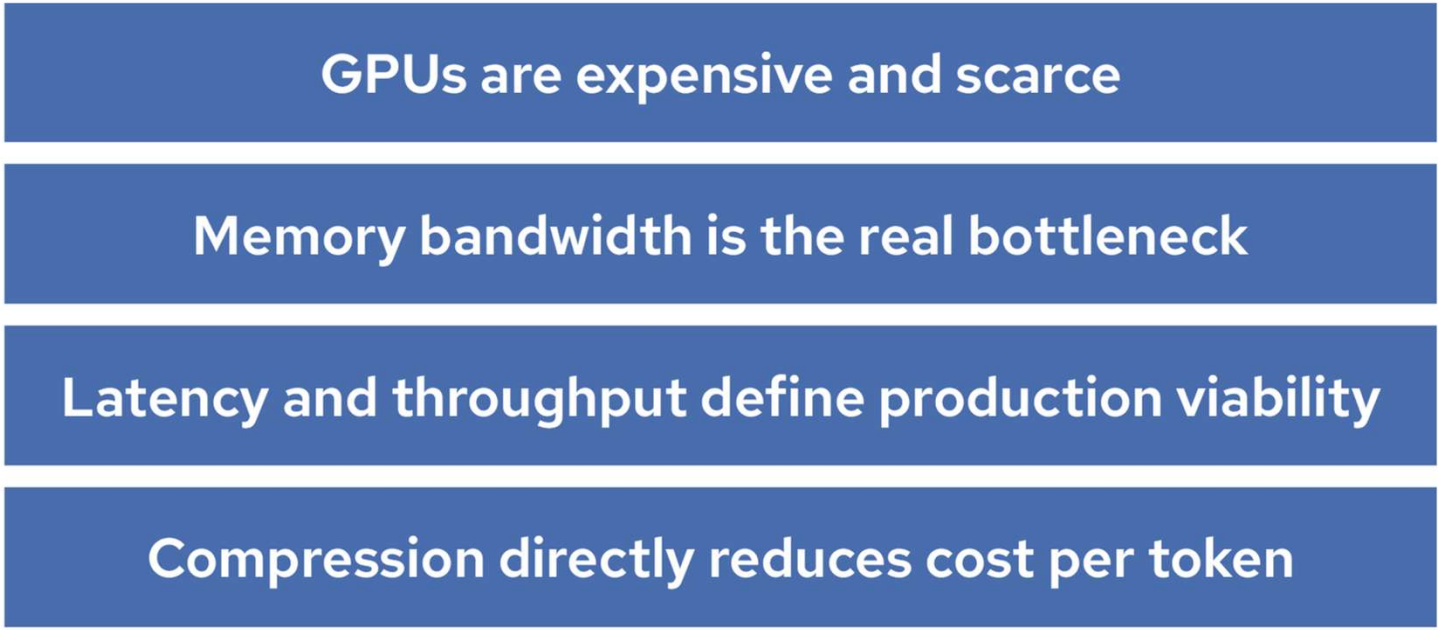
Source: Redhat



Quantization & Model Compression

Model compression

The drivers for model compression (also known as optimization and/or quantization):



Source: Redhat

Hardware, Cost, Performance Tuning

Models come in different sizes. The bigger the model (measured in billions of parameters), the more performant and detailed its responses can be. But bigger models need more powerful GPUs and more memory, which can make them slower or more expensive to run.

The biggest bottleneck for loading and running a model is **GPU memory**. An easy way to think of it is:

Number of parameters x 4 = GPU memory needed

For example, a 3B parameter model requires ~12GB of GPU memory to load and run.

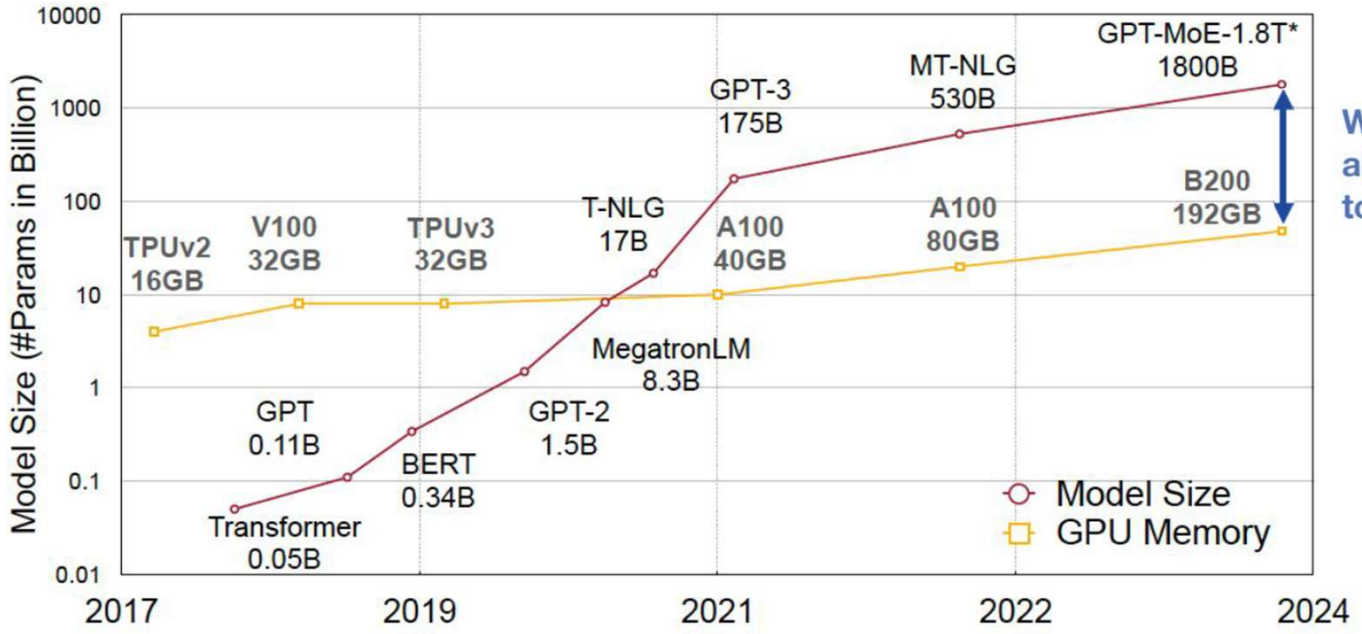
Model Size	Parameters	GPU Requirement	Notes
<3B	Small	8–12GB VRAM	Lightweight, fast, and easy to deploy
7B–13B	Medium	≥24GB VRAM or quantized*	Balanced power and cost
>30B	Large	Multi-GPU or high-end cards	Better at understanding context but slower and costly

Source: Redhat

Note: This is for FP32, but most models now are FP16 – but w still budget 4x because of KV cache & activations

GPU Memory

The growth of GPU memory is not keeping up with the growth of models.



We need efficient algorithms and systems to bridge the gap.

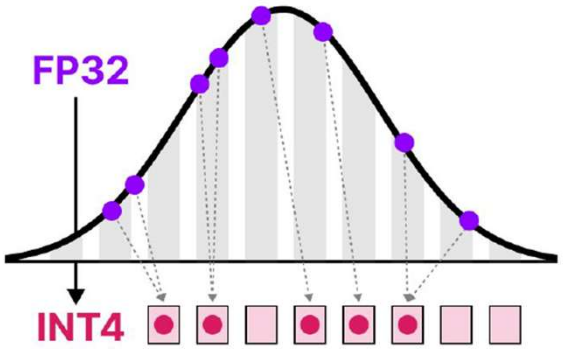
Source: Redhat

Model compression types

Compression leads to cost saving – reducing memory and compute requirements - without performance loss

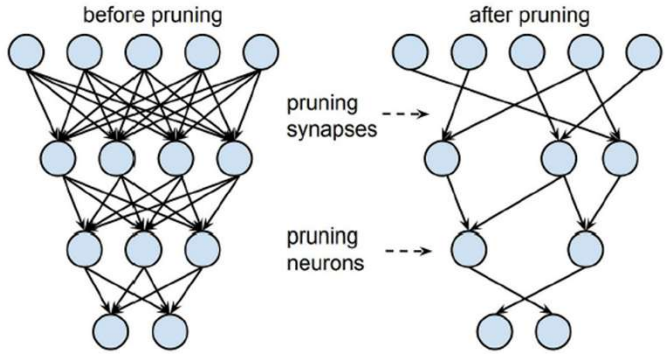
Quantization

Quantization is the process of reducing a model's numerical precision (e.g. from 32-bit to 4-bit) to shrink memory footprint and speed up inference with **minimal accuracy loss.**



Pruning & Sparsity

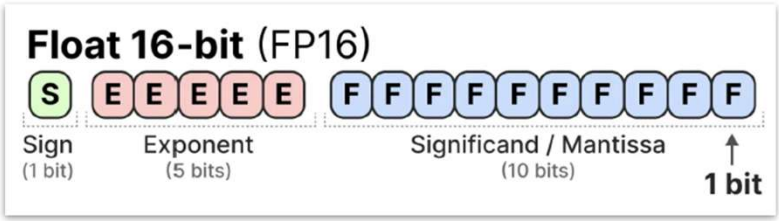
Pruning is the process of removing redundant or less important weights/neurons from a model to reduce size and improve efficiency with **minimal impact on accuracy**.

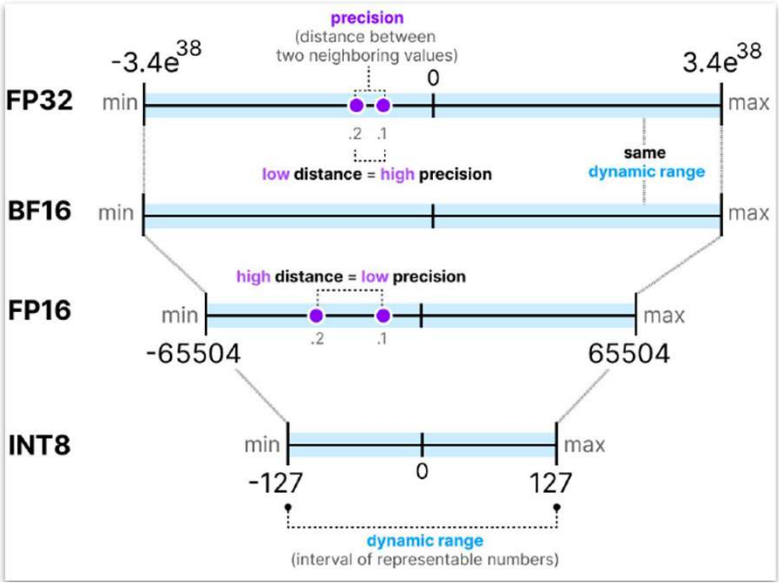


Source: Redhat

Compression

- LLMs are a series of matrix multiplications of trained ”parameters”
- Each ”parameter” is a value represented by a series of some number of bits
- The more bits used to represent a parameter the more detail we can express:
  - Dynamic range (min—max value)
  - Precision (values close to zero)





Source: Redhat

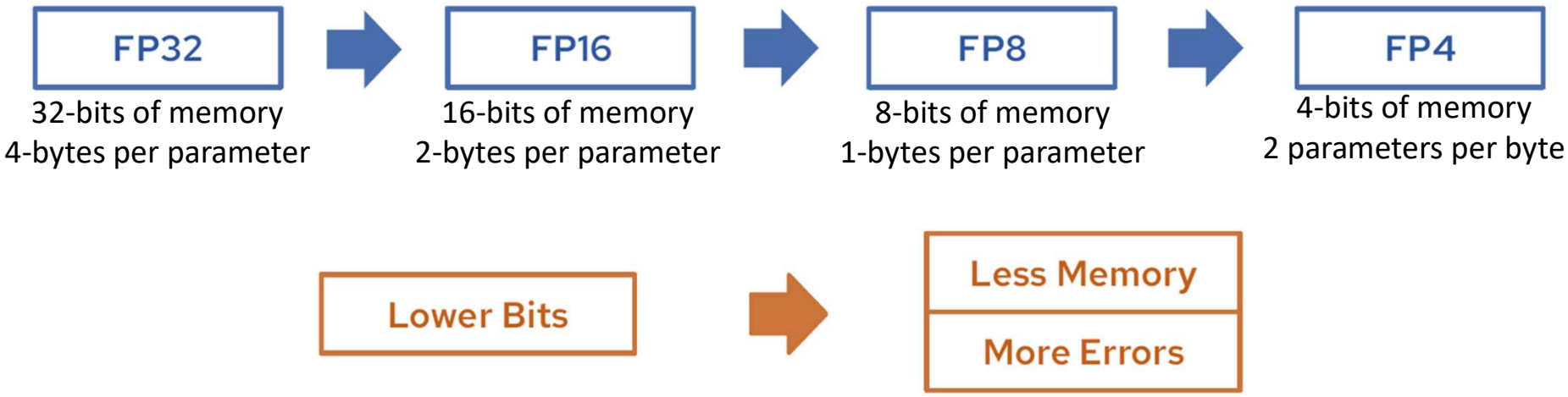
Pruning & Sparsity

Pruning	Removes weights or neurons Structured vs unstructured pruning Often combined with quantization
Sparsity	Many weights are zero **Requires kernel support** Hardware dependent benefits

Source: Redhat

Quantization

Quantization replaces parameter floating-point values with lower-precision values (representations), resulting in a much smaller model. A smaller model means lower cost and higher throughput, but at some impact on accuracy.

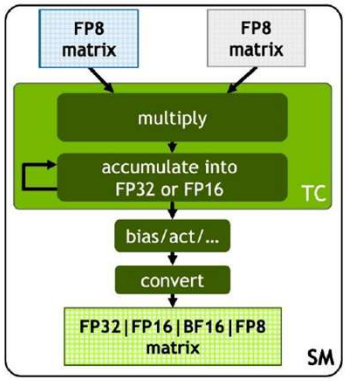


Source: Redhat

Quantization in vLLM

Low precision can be used to store and compute models e.g. FP8, INT8, FP4

- **Weight Quantization**
  Reduced storage & memory footprints
  e.g. 100B model (BFloat16 —> 200GB / FP8 —> 100GB).
- **Faster linear layers**
  Compute speedups.
- **Reduced KV cache footprint & faster attention**
  Crucial for long context workloads.



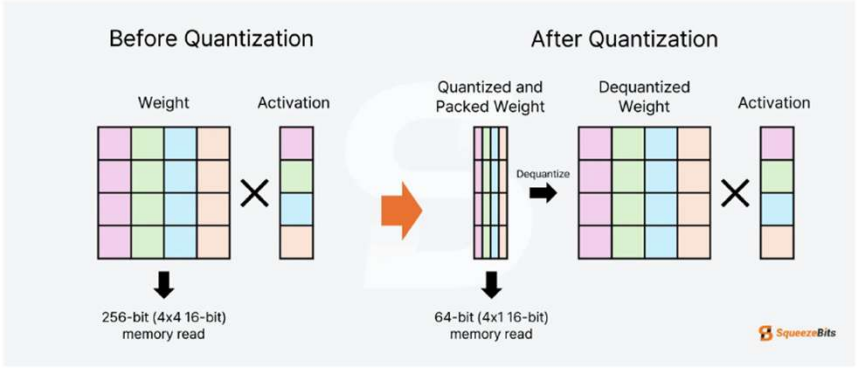
	B200	H100	A100 (80GB)
FP4 Tensor	9 PFLOPS	N/A	N/A
INT8/FP8 Tensor	4500 T(FL)OPS	1980 TOPS	624 TOPS
FP16 Tensor	2250 TFLOPS	990 TFLOPS	312 TFLOPS
TF32 Tensor	1100 TFLOPS	495 TFLOPS	156 TFLOPS
FP64 Tensor	40 TFLOPS	67 TFLOPS	19.5 TFLOPS

Source: Redhat

Weight Quantization: W8A16, W4A16, WNA16

- **Lower-bit precision**
  Reduce GPU RAM requirements by squeezing the parameters into lower-bit precisions.
- **Reduced data transfer**
  Reduce data movement at the expense of some extra compute to upconvert for each forward pass.
- **Lower-bit precision**
  Weight-only quantization is not sufficient for speedups once under load (long prefills or batching).
- **Low-precision tensors**
  Quantizing weights and activations means we can finally use low precision tensor cores.

Implementation	Volta	Turing	Ampere	Ada	Hopper	AMD GPU	Intel GPU	x86 CPU
AWQ	×	✅	✅	✅	✅	×	✅	✅
GPTQ	✅	✅	✅	✅	✅	×	✅	✅
Marlin (GPTQ/AWQ/FP8)	×	×	✅	✅	✅	×	×	×
AQLM	✅	✅	✅	✅	✅	×	×	×
bitsandbytes	✅	✅	✅	✅	✅	×	×	×
DeepSpeedFP	✅	✅	✅	✅	✅	×	×	×
GGUF	✅	✅	✅	✅	✅	✅	×	×



Source: Squeezebits, Redhat



Latency & Throughput

Performance Key Metrics

Let's consider how we measure a model's responsiveness and efficiency.

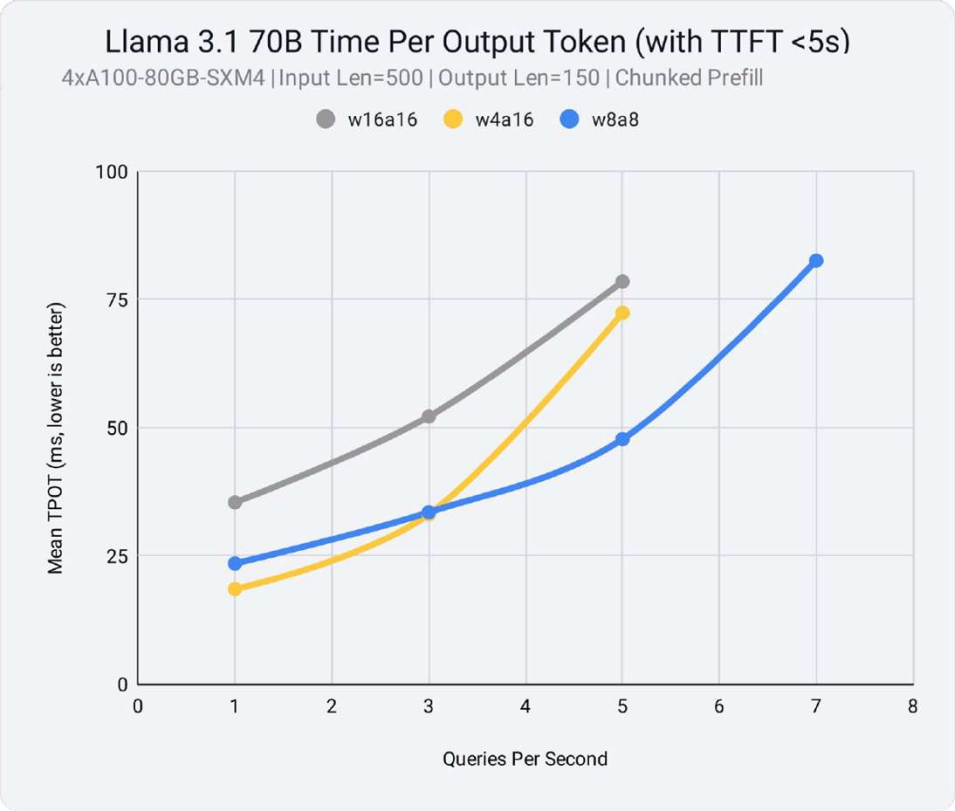
The following metrics indicate how fast a model can start generating text, how quickly it produces each word, and how much resource it uses.

Metric	Meaning
TTFT	Time to First Token – how quickly the model starts giving its first token after you ask something.
TPOT	Time Per Output Token – the speed at which each next token is generated once the response has started.
Throughput	How many requests the system can handle at the same time without slowing down.
VRAM Usage	The amount of GPU memory the model needs (bigger models or longer conversations need more memory).

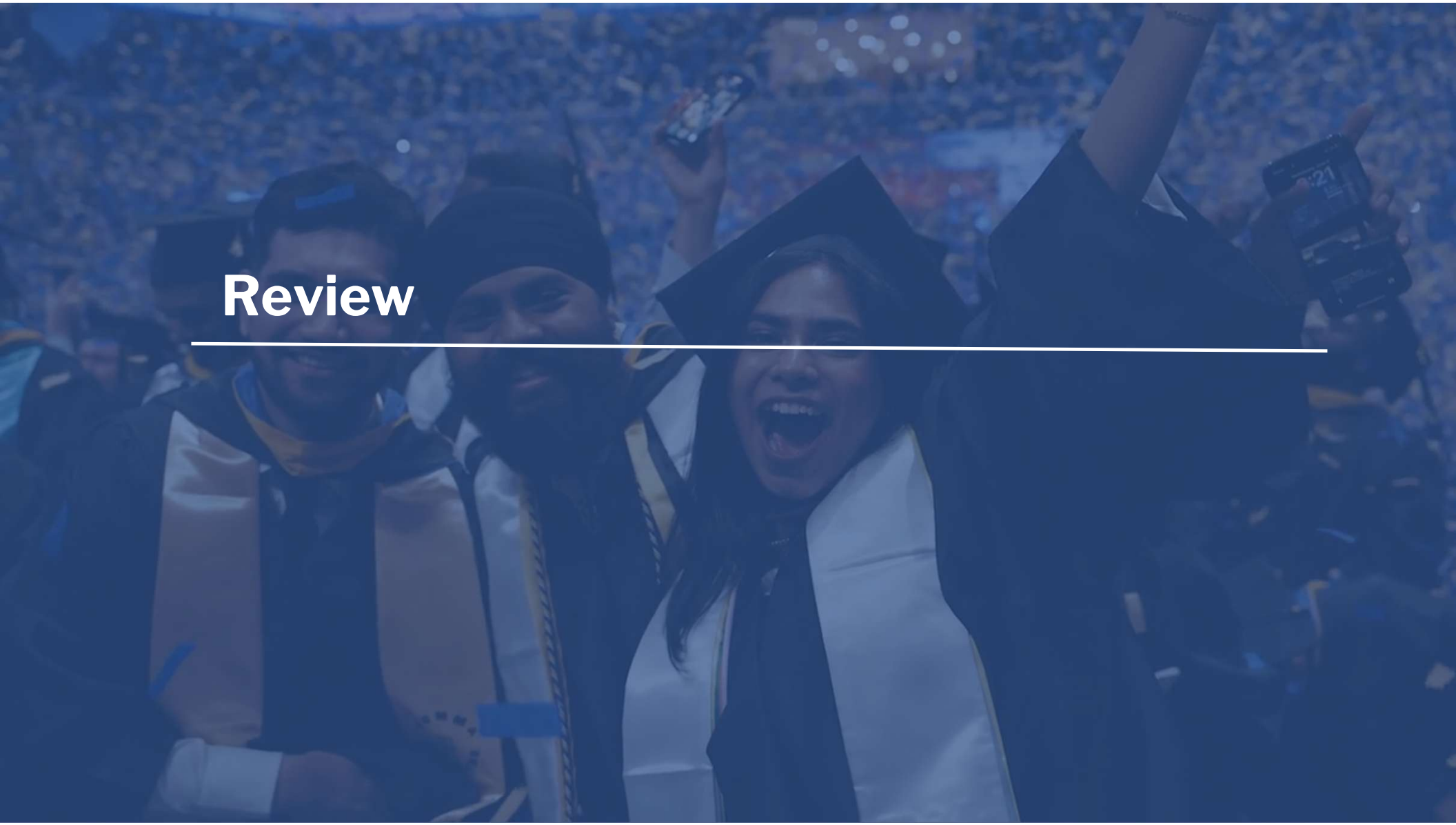
Source: Redhat

Inference throughput

Quantization enables more tokens for the same hardware.



Source: Redhat



Review

This week we covered

Lesson Objectives/Topics

- **Foundational**
  - Model inference and serving
  - Quantization and model compression
  - Latency and throughput


CUNY | School of
Professional Studies